

Advanced Manual **Smart Contract Audit**

September 30, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

Nlyra

Real Yield Staking v2, PositionMarket.sol

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	3	0
● Low-Risk	3	0	3	0
● Medium-Risk	0	0	0	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

Nlyra

Website	nlyra.xyz
---------	-----------

Blockchain	Robinhood Chain (Arbitrum Orbit, chainId 4663)
------------	--

Source Code	github.com/NlyraProject/nlyra-platform, staking-v2/src (commit 19880c0)/PositionMarket.sol (217 lines)
-------------	--

Contract Address	0x3BcA70536aC7FfB44023e971d204dAb7c23E95D7
------------------	--

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Audit Summary	2
Audit Scope	3
Audit Method	3
Automated Vulnerability Check	3
Manual Code Review	3
Used tools	3
Table of Contents	4

● Nobody Can Interfere With A Listing Or Take A Seller's Money

Informational - Does not compromise the functionality of the contract in any way **[Status: Acknowledged]**

Error Code	Description
PM-I1	The contract has no owner and no settings, the 0.5 percent commission is fixed in the code, and we found no way for the deployer, the staking contract's owner or anyone else to remove a listed position or take a seller's proceeds. Sellers are paid by claiming rather than being sent money during a sale, so a sale can never fail because a seller's wallet cannot receive funds, and on creation the contract checks that the fee destination it was given genuinely belongs to the staking contract it was pointed at.

Code Snippet

```
uint256 public constant FEE_BPS = 50; // 0.5%, fixed forever
// no owner, no setters, no upgrade path

// the fee destination is cross checked against the staking contract
address splitter = RealYieldStaking(staking).feeSplitter();
if (splitter == address(0))
    || NlyraFeeSplitter(payable(splitter)).STAKING() != staking
    revert BadStaking();
```

Recommendation

No change is needed. This is a good baseline and it is worth stating clearly in public material, because buyers and sellers of positions are trusting this contract with real money.

● A Position Number Is Stored In A Smaller Field Than It Is Looked Up With

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
PM-I2	A listing stores the position number in a field that holds values up to about four billion, while the record tracking which listing belongs to which position is filed under the full sized number. These can never disagree today, because the staking contract caps positions at 32 and anything outside that is rejected earlier, but the guarantee lives in a different contract rather than in this one.

Code Snippet

```
uint32 positionId;    // stored narrowed
_listings[listingId] = Listing(msg.sender, expiry, uint32(positionId), ...);

// but filed under the full sized number:
activeListing[msg.sender][positionId] = listingId;

// safe only because the staking contract caps positions at 32
```

Recommendation

Add an explicit check that the position number is within the supported range when a listing is created, so the guarantee lives in this contract rather than depending on a constant in another one. This is a small change and removes a hidden link between the two contracts.

● The Commission Is Rounded Up, So Very Small Listings Pay Everything In Fees

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
PM-I3	The commission always rounds up, which is the correct direction and invisible at any realistic price. At the very bottom of the range it becomes absolute: a listing priced at a single unit pays that entire unit as commission and leaves the seller with nothing, and the contract accepts such a listing without complaint. Nobody would price a real position this way, so this is about behaviour at the extreme rather than a practical concern.

Code Snippet

```
// rounds up, which is the right direction
uint256 fee = (uint256(l.price) * FEE_BPS + BPS - 1) / BPS;
uint256 net = l.price - fee;

// at a price of 1 the seller receives 0
```

Recommendation

Optionally set a minimum listing price so a seller cannot accidentally list at a price that leaves them nothing. The current behaviour is not dangerous, so this is a polish item rather than a fix.

● A Seller Can Make A Buyer's Purchase Fail At The Last Moment

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PM-L1	A sale only completes if the position is still exactly as advertised and still offered to this marketplace, which is the right design and is what stops a buyer being handed something different from what they agreed to. The side effect is that a seller can make a pending purchase fail at the last moment by withdrawing the offer, removing the listing or extending the lock. The buyer keeps their money, so the only loss is the transaction fee, but a seller who watches for incoming purchases can repeatedly waste a specific buyer's fees.

Code Snippet

```
// all of these are enough to make a pending purchase fail:
//   staking.cancelPositionOffer(positionId)
//   market.cancel(listingId)
//   staking.extendLock(positionId, tier)    // changes the snapshot

if (p.amount != l.amount || p.unlockTime != l.unlockTime
    || p.tier != l.tier) revert PositionChanged();
```

Recommendation

This cannot be removed without weakening the protections that make the marketplace safe, so the answer is to reduce the damage rather than prevent the behaviour.

Interfaces should check that a listing is still purchasable immediately before submitting, using the view function this contract already provides, and should warn users that a seller can withdraw at any point. A seller who does this repeatedly is visible on chain and can be excluded from the front end.

● A Listing Keeps Its Original Price Even As The Lock It Sells Runs Down

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PM-L2	A listing records its price and the position's details when created and stays valid until an expiry the seller chooses, which can be set far into the future. Nothing links the two, so a position listed with 29 days of lock remaining can still be bought weeks later at the original price with only days left, because the safety check only requires the position to be unchanged. All of it is visible on chain, so the risk falls on buyers relying on an interface that checks only whether a listing is currently purchasable.

Code Snippet

```
// the listing stores a price and an expiry chosen by the seller
if (expiry <= block.timestamp) revert BadExpiry(); // no upper bound

// buying only requires the position to be unchanged and still locked
if (block.timestamp >= p.unlockTime) revert NotLocked();
// nothing requires a minimum remaining lock period
```

Recommendation

Show the remaining lock time prominently next to the price wherever listings are displayed, and consider having the interface hide or flag listings whose remaining time has fallen well below what it was when they were created. If you would rather enforce it in code, allowing a seller to set a minimum remaining lock period alongside the expiry would let the listing lapse on its own.

● A Buyer Whose Position Slots Are Full Cannot Complete A Purchase

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PM-L3	A purchase delivers the position straight into the buyer's wallet, which needs one of their 32 slots free, so a buyer already at the limit cannot buy. Read together with the finding in the staking report, where anyone can push unwanted positions onto a wallet without consent, this means a third party can cheaply and repeatedly block a chosen person from buying here. The view that tells an interface whether a listing can be bought deliberately ignores the buyer's slots, so there is no warning until the transaction fails.

Code Snippet

```
// delivery needs a free slot in the buyer's wallet
newId = STAKING.acceptPositionTo(l.seller, l.positionId, msg.sender);
// reverts with TooManyPositions if the buyer holds 32 already

// and the buyability view does not look at the buyer at all:
/// No mira el cupo de 32 del comprador.
```

Recommendation

Have the interface check the buyer's free slots before offering a purchase, so the problem is explained rather than appearing as a failed transaction. The underlying fix belongs in the staking contract and is covered in that report: positions should not be deliverable to a wallet that has not agreed to receive them.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.